- The ejection occurs from the order of data being added.
- From the point of view of the terms of implementation and performance this technique is fast but not smart.

LIFO (last in first out)
- This is the opposite of FIFO -- once the cache is full, the last added item is removed first.
- The ejection occurs in the reverse of the order of data being added.
- This technique, too, is fast but not smart.

LFU (least frequently used)
- Here the data keeps track of how frequently it has been used from the cache, and the count is maintained.
- Once the cache is full, the lowest count data gets removed first.

LRU (least recently used)
- Initially any read data is added to the cache, but when the cache is full it frees up the least recently used data.
- Here, each time the data is read from the cache it's moved to the top of the queue, increasing its significance.
- This is a fast and most commonly used algorithm.
- 'Temporal Locality' uses a similar concept while saving the recently used instructions in cache memory, as there are high chances of these being used again.

LRU2 (least recently used twice)
- Two caches are maintained here — the item is added to the main cache only when it is accessed a second time.
- Once the cache is full, the least recently accessed item is removed.
- This is complex and more space is required, as there are two caches and the count of accesses is maintained.
- The advantage is the main cache holds the most frequently and recently accessed data.

2Q (two queue)
- Here, as well, two queues — one small and one large — are maintained.
- First, accessed data is added to the smaller LRU queue.
- If the same data is accessed a second time, it is moved to the larger LRU and removed from the first queue.
- Performs better than LRU2 and makes it adaptive.

MRU (most recently used)
- Quite opposite to LRU, here the most recently accessed data is removed from the cache.
- This algorithm leans towards the older data, which is more likely to be used again.

STBE (simple time based expiration)
- Here, once the data is added to the cache, its lifetime tickers get started.
- Data is removed from the cache after a certain time period, e.g., at 5.00 pm or a particular date or time.

ETBE (extended time based expiration)
- Data from the cache is removed after a certain time period.
- Here the time to evict data is configurable; e.g., five hours from now, or every ten minutes.

SLTBE (sliding time based expiration)
- The timeline of the data extends after being accessed from the cache.
- In this algorithm, the most recently accessed data gets more time to stay in the cache.

WS (working set)
- This is similar to LRU, but here a flag is created for each access in the cache.
- Periodically, the cache is checked, considering the recently accessed data in the working sets.
- Non-working set data is removed when the cache is full.

RR (random replacement)
- Here, data is picked randomly from the cache and replaced with the newly accessed data.
- This approach does not keep track of the history, resulting in less overhead, but the results are not guaranteed.

LLF (lowest latency first)
- This algorithm keeps track of download latency time.
- The data with the least downloaded latency time is evicted first, as it can be quickly retrieved again.
- Here, the advantage is when complex data needs to be retrieved, it can be easily referred from the cache.

LRD (least reference density)
- The object with the least reference density is removed when the cache is full.
- A global reference counter is maintained, containing the sum of all references in the cache.
- The reference density (RD) is calculated based on:
  - Object's reference counter (RC) meaning the number of times it has been accessed
  - Total number of all the references (GC)
  - Each object has an arrival timestamp (AT), which is the current GC value when it's been added to the cache
- The reference density (RD) is computed as the ratio between the object's reference counter (RC) and the number of references added since the object was included into the cache (GC – AT):
  - RC(i) / (GC - AT(i)).